

Adapter

Les patrons structuraux

Les **patrons structuraux** visent à organiser les objets dans des structures plus complexes, comme des hiérarchies, des compositions ou des relations d'adaptation.

Nous allons aborder les patrons suivants :

- **L'adaptateur** : permet de faire fonctionner ensemble des interfaces ou des prototypes a priori incompatibles.
- **Le composite** : représente des objets sous forme d'arbre, permettant de traiter uniformément les objets individuels (feuilles) et les groupes d'objets (nœuds de l'arbre).
- **Le décorateur** : permet d'ajouter dynamiquement des fonctionnalités à un objet.

L'Adapter

L'**Adapter** est un patron de conception structurel permettant de faire communiquer des objets aux interfaces incompatibles, sans modifier le code existant.

Problématique

Imaginons un logiciel de diagnostic pour téléphones utilisé dans des centres de réparation. Ce logiciel s'appuie sur une interface **Charger** et une classe **StandardCharger** qui implémente cette interface pour charger des téléphones implémentant **Chargeable**.

Code initial

```
interface Charger {  
    void connectPhone();  
}  
  
interface Chargeable {  
    void charger();  
}
```

```

}

class SimplePhone implements Chargeable {
    public void charger() {
        System.out.println("Charging SimplePhone ...");
    }
}

class StandardCharger implements Charger {
    Chargeable phone;

    public StandardCharger(CHargeable phone) {
        this.phone = phone;
    }

    public void connectPhone() {
        phone.charger();
    }
}

public class App {
    public static void main(String[] args) {
        Chargeable simplePhone = new SimplePhone();
        StandardCharger standardCharger = new StandardCharger(simplePhone);
        standardCharger.connectPhone();
    }
}

```

Le problème survient lorsque nous devons ajouter un **USBCSmartphone**, qui utilise une méthode `chargeWithUSBCcable()` au lieu de `charger()`.

Code du nouveau téléphone incompatible

```

public class USBCSmartphone {
    public void chargeWithUSBCcable() {
        System.out.println("Charging USB-C phone");
    }
}

```

```
}  
}
```

Ce type de téléphone ne peut pas être pris en charge par **StandardCharger** sans modification.

Solutions alternatives

Solution 1 : Modifier **USBCSmartphone** pour implémenter **Chargeable**

```
public class USBCSmartphone implements Chargeable {  
    public void chargeWithUSBCable() {  
        System.out.println("Charging USB-C phone");  
    }  
  
    @Override  
    public void charger() {  
        chargeWithUSBCable();  
    }  
}
```

Inconvénients :

- Nécessite l'accès au code source de **USBCSmartphone**.
- Duplique la logique (**chargeWithUSBCable()** et **charger()** font la même chose).
- Impacte le code client existant.

Solution 2 : Modifier **StandardCharger** pour gérer **USBCSmartphone**

```
class StandardCharger {  
    public void connectUSBPhone(USBCSmartphone usbCPhone) {  
        System.out.println("Connecting USB-C Phone");  
        usbCPhone.chargeWithUSBCable();  
    }  
}
```

Inconvénients :

- StandardCharger devient spécifique à certains téléphones.
- Manque de généricité.
- Maintenance difficile (ajout de nouvelles méthodes pour chaque type de téléphone).

Solution optimale : l'Adapter

Nous allons créer une classe `USBCAdapter` qui implémente `Chargeable` et adapte `USBCSmartphone`.

```
public class USBCAdapter implements Chargeable {
    private USBCSmartphone usbCPhone;

    public USBCAdapter(USBCSmartphone usbCPhone) {
        this.usbCPhone = usbCPhone;
    }

    @Override
    public void charger() {
        usbCPhone.chargeWithUSBCcable();
    }
}
```

Code de l'application avec l'Adapter

```
public class App {
    public static void main(String[] args) {
        Chargeable simplePhone = new SimplePhone();
        StandardCharger standardCharger = new StandardCharger(simplePhone);
        standardCharger.connectPhone();

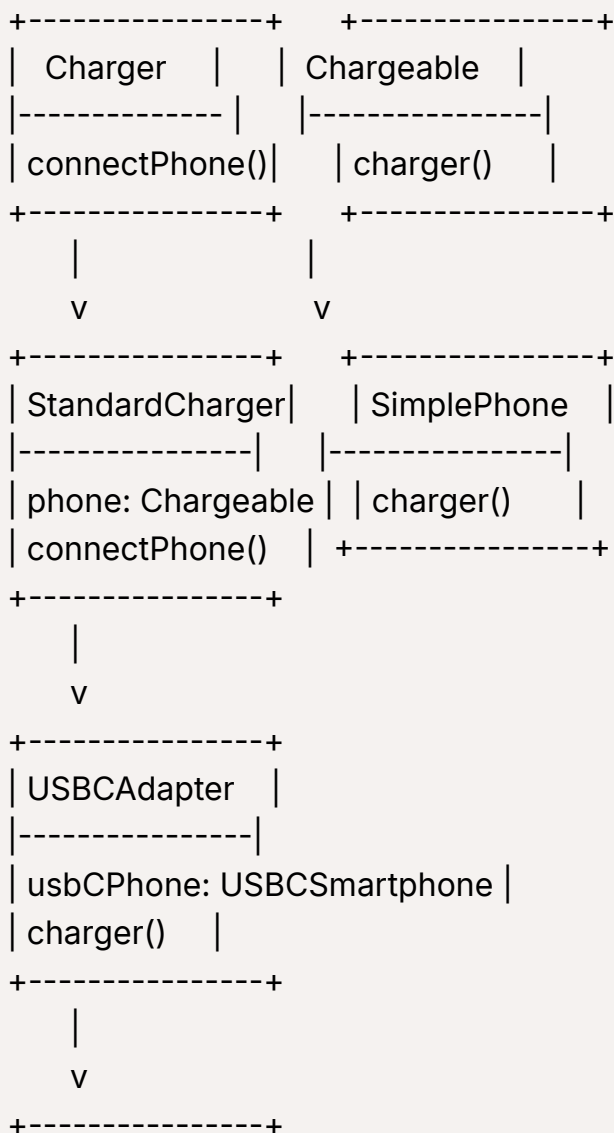
        USBCSmartphone usbCPhone = new USBCSmartphone();
        Chargeable adaptedPhone = new USBCAdapter(usbCPhone);
        standardCharger = new StandardCharger(adaptedPhone);
        standardCharger.connectPhone();
    }
}
```

```
}
}
```

Avantages du patron Adapter

- **Garde le code existant intact** : aucune modification n'est nécessaire sur `StandardCharger` ou `USBCSmartphone`.
- **Facilite l'extension** : tout nouveau type de téléphone peut être ajouté en créant un nouvel adaptateur.
- **Améliore la maintenabilité** : évite la prolifération de méthodes spécifiques dans `StandardCharger`.

Schéma UML



```
| USBCSmartphone |  
|-----|  
| chargeWithUSBCcable() |  
+-----+
```

Ce patron permet d'ajouter de la compatibilité sans modifier le code existant et est idéal pour les logiciels en évolution.